

队伍编号	202503861
选题	A 题

康养资源动态优化模型构建及策略研究——以太原市为例

摘 要

随着我国人口老龄化加速，康养城市建设已经成为应对人口结构变化的重要举措。但当前面临康养资源分布不均、技术应用不足等问题，制约了康养服务水平的提升。本文聚焦于康养城市建设的核心问题，构建了**资源供需适配量化评估模型**、**综合评价模型**以及**多目标优化配置模型**，并融合**粒子群优化法**、**熵权法**、**灰色关联分析**等方法来解决相关问题并给出了优化策略。

针对问题一，为分析城市内不同区域康养资源分布与居民健康需求的合理性，本文建立了**综合指标构建法**与**资源供需适配量化评估模型**，通过**熵权法**构建资源指数与居民健康指数，计算匹配度来识别失衡区域，以太原市为例，计算得阳曲县（1.55）和小店区（0.4）的匹配度差异很大，呈现出“郊区过剩，城区短缺”的特征，因此引入**粒子群优化算法**动态调整资源分配，优化后结果表明，在总资源守恒下，能够实现各区域康养资源供需趋于平衡，明显改善了供需失衡问题。

针对问题二，为系统评估当前城市康养水平，本文构建了**康养城市综合评价模型**，从资源、健康、环境、经济四个维度构建了13项指标体系，采用**熵权法**赋权值并结合**灰色关联分析法**，评估各城市与理想康养状态的接近程度。结果显示，娄烦县（0.69）、晋源区（0.68）因绿地资源丰富和健康满意度高，综合得分靠前；市中心城市因医疗设施人均密度低，空气质量差排名靠后，需重点提升资源供给与污染治理。

针对问题三，在有限资源条件下，为实现高效配置康养资源，本文构建了**多目标优化配置模型**，以最大化服务覆盖率和最小化建设成本为目标，结合**粒子群优化算法**求解设施选址方案，并给出具体的实施策略。结果表明，迎泽区因经济基础强（人均GDP权重5.07%）、服务辐射能力优为最优选址，并建议引入动态监测平台，实时调整服务半径（ $\pm 2\text{km}$ 浮动）和设施容量（ $\pm 15\%$ 弹性阈值），以应对需求波动。

整体来看，本文通过多模型融合与算法优化，实现了康养资源的量化评估与科学配置，模型经统计检验和敏感性分析验证有效，提出的实施策略也为康养城市的建设提供了可参考的解决方案。

关键词：**熵权法**、**灰色关联分析**、**粒子群优化算法**、**多目标优化配置模型**、**资源供需适配量化评估模型**

一、问题重述

1.1 问题背景

随着我国老龄化的进程不断加快，老年人面临的健康和养老等问题也越来越突出^[1]。2025 年全国两会提出推动康养产业转型、发展智慧养老等政策方向，地方城市也通过盘活闲置资源、优化设施布局等措施积极探索和实践。然而，康养资源分布不均、智慧养老技术应用不足等问题依然突出，制约了康养服务的公平性与可持续性。

在此背景下，如何精准评估康养资源布局、构建综合评价体系、优化资源配置效率，成为建设康养城市的关键。一方面，资源分配不均导致部分区域服务供给过剩与短缺并存，影响老年群体生活质量；另一方面，缺乏系统性评价模型难以指导政策精准发力。因此，量化分析资源分布、制定优化策略，是落实国家战略的必然要求，更是实现“老有所养、老有所依”社会目标的关键途径。

1.2 问题重述

问题 1：在康养城市建设中，需考虑城市内不同区域的资源分布情况和居民的健康状况，包括该城市内有多少家养老机构、人均公园绿地面积为多少、居民的健康满意度等方面，以此来分析城市的康养资源分布是否合理，并针对分布不均的问题，提出如何优化康养资源布局。

问题 2：题目要求构建一个综合评价模型，从康养资源、健康状况、环境质量、经济发展等多维度入手，给当前城市的康养水平做出一个全面评估。并通过分析其数据，找出其优缺点，说明在哪些方面做的较好和较差，最后针对性的给出具体的改进措施，指明其康养建设的道路。

问题 3：在资源有限的情况下，如何高效配置康养资源，即要让服务覆盖更多人且不浪费资源，又要控制建设成本，还要考虑设施选址、数量和服务范围，因此需构建一个康养资源优化配置模型，然后根据模型结果以及问题 1 和问题 2 的结论，给出具体的实施策略，包括政策、资金投入、技术创新等多方面。

二、问题分析

2.1 问题 1 的分析

题目涉及某城市不同区域内康养资源与需求的匹配情况。已知条件中给出各区域多项康养相关数据，如医疗设施数量、健康满意度等。整体来看，需评估现有康养资源分布的合理性，再着重解决如何优化康养资源布局的问题。基于此，本文建立了**综合指标构建法与资源供需适配量化评估模型**，采用熵权法确定各项指标权重，计算初始匹配度，分析是否合理，引入**粒子群优化算法**（PSO）进一步优化，设定粒子参数、以匹配度为适应度函数进行迭代寻优，得到更优的资源匹配方案，提升区域康养资源利用效率

2.2 问题 2 的分析

问题 2 聚焦于构建**康养城市综合评价模型**，从康养资源丰富度、居民健康状况、环境质量、经济发展水平等多个维度评估当前城市康养水平的现状，探寻其优势与短板。

本文着力于找出影响城市康养水平的关键因素，建立四级指标体系，采用熵权法确定各项指标权重，并计算综合得分，同时引入灰色关联分析法量化各区域与“理想康养状态”的差距，精准定位其优势与不足之处

2.3 问题 3 的分析

题目要求建立康养资源优化配置模型并给出改进建议。在资源有限的约束条件下，合理选址康养设施，服务尽可能多的老年人口，这需满足在成本、容量、服务半径等现实约束的同时，寻求服务覆盖率与建设成本之间的平衡，实质上是一个设施选址与服务覆盖最大化的多目标优化问题。为此，本文建立了多目标动态粒子群优化模型，设置 0/1 布点变量 x 与服务分配变量 y ，并引入粒子群优化算法（PSO），输出多组帕累托最优解。结合前两问的结果，制定差异化策略，给出具体的实施策略

三、模型假设

1. 康养资源总量短期内保持不变
2. 居民健康数据真实可靠，且与资源需求线性相关。
3. 设施服务半径内人口需求均匀分布。
4. 经济发展水平与康养投入能力正相关。

四、符号说明

符号	说明
R_i	资源指数
H_i	居民健康需求指数
ρ_i	资源匹配度
$\vec{x}$	每一个粒子（每一种资源重新配置方案）
$v_i^{(t+1)}$	粒子的速度
$x_i^{(t+1)}$	粒子的位置
x_{ij}	第 i 个城市在第 j 项指标的原始值
z_{ij}	归一化后的值（统一到 $[0, 1]$ 区间）
p_{ij}	归一化后的比例
e_j	信息熵
d_j	熵权系数
w_j	主客观综合指数
S_i	综合评分
X_i	第 i 项指标

五、模型建立与求解

5.1 数据收集与整理

我们选择太原市作为研究对象，因其康养指数显著低于全国领先城市，且在同类省

会城市中排名靠后，说明其康养发展存在许多短板。通过分析其数据与区域特征，能够为优化康养资源配置和制定针对性提升策略提供科学依据

同时，考虑到康养资源是空间性资源，不可能统一规划到整个城市的每一个角落。因此，我们按照太原市划分的十个区作为本文所研究的区域范围，并对每一个区分别收集数据。

本文数据主要来源于太原市统计局公开的数据，数据涵盖了康养资源丰富度、居民健康状况、环境质量和经济支撑能力这四个维度共 13 项指标，包括医疗设施数、养老机构数、平均寿命等。

5.2 问题 1 模型的建立与求解

5.2.1 问题 1 模型的建立

基于问题 1 要求，本文建立了**综合指标构建法与资源供需适配量化评估模型**来实现对太原市内不同区域康养资源的全面评估。通过归一化与熵权法分别构建资源指数和健康指数的综合指标体系，将其比值作为资源供需适配度，进而识别当前康养资源分布失衡区域。在此基础上，引入粒子群优化算法（PSO）模拟人类智慧调整资源的过程，在保持总资源不变的前提下，重新分配资源以尽量平衡各区域资源匹配度，解决康养资源分布不均衡的问题，从而提升太原市康养服务的公平性和可达性。

（1）综合指标构建法

康养资源的分布评价中涉及多个指标，如养老机构数量，公园绿地面积等，这些指标量纲与取值范围差异巨大。若直接用于分析，数值大的指标会主导结果，从而掩盖其他指标的影响。通过归一法将所有数据压缩至 $[0, 1]$ 区间，使不同类型的康养资源具有可比性，为后续分析奠定基础。

$$x_{ij}^* = \frac{x_{ij} - \min(x_j)}{\max(x_j) - \min(x_j)} \quad \text{式 1}$$

$$X_{ij}^* = \frac{\max(x_j) - x_{ij}}{\max(x_j) - \min(x_j)} \quad \text{式 2}$$

其中， x_{ij} 表示第 i 个区域第 j 个指标的原始值， x_{ij}^* 表示归一化后的值， $\min(x_j)$ 、 $\max(x_j)$ 表示第 j 个指标的最大值和最小值。

在康养资源评价中，传统主观赋权法可能因专家个人认知，过度重视养老机构的数量，而忽视康复设施的完备程度等其他重要指标。熵权法依据信息熵理论，从数据本身出发，采用熵权法可以充分利用客观数据所提供的信息来确定客观权重，去除主观性影响^[2]。

$$p_{ij} = \frac{x_{ij}^*}{\sum_{i=1}^n x_{ij}^*} \quad \text{式 3}$$

其中， p_{ij} 表示指标占比， n 表示太原市的区域数量。

$$e_j = -\frac{1}{\ln n} \sum_{i=1}^n p_{ij} \ln p_{ij} \quad \text{式 4}$$

其中, e_j 表示指标熵值, 反映指标信息的无序程度, $e_j \in [0, 1]$ 。

$$d_j = 1 - e_j \quad \text{式 5}$$

其中, d_j 表示指标差异系数, 差异系数越大, 指标对评价的重要性越高。

$$w_j = \frac{d_j}{\sum_{j=1}^m d_j} \quad \text{式 6}$$

其中, w_j 表示指标权重, m 表示指标数量。

基于此我们构建出资源指数 R_i 和健康指数 H_i 。

$$R_i = \alpha_1 \cdot \frac{M_i}{P_i} + \alpha_2 \cdot \frac{E_i}{P_i} + \alpha_3 \cdot \frac{G_i}{A_i} + \alpha_4 \cdot \frac{C_i}{P_i} \quad \text{式 7}$$

$$H_i = \beta_1 \cdot L_i - \beta_2 \cdot D_i + \beta_3 \cdot S_i \quad \text{式 8}$$

其中, M_i 表示区域 i 的医疗设施数, E_i 表示区域 i 的养老机构数, G_i 表示区域 i 的绿地面积, C_i 表示区域 i 的文化设施数, P_i 表示区域 i 的人口数, A_i 表示区域 i 的面积, L_i 表示平均寿命, D_i 表示慢性病患率, S_i 表示居民健康满意度。

将归一法和熵权法应用于康养资源分布评价模型构建, 形成了完整且严谨的综合评估体系。

(2) 资源供需适配量化评估模型

基于上述资源指数 R_i 和健康指数 H_i , 构建资源供需适配量化评估模型。

$$\rho_i = \frac{R_i}{H_i} \quad \text{式 9}$$

(3) 粒子群优化算法 (PSO) 优化分析

在保持总资源不变的前提下, 我们引入以下目标函数:

$$f = \sum_{i=1}^n |\rho_i - 1| = \sum_{i=1}^n \left| \frac{R_i}{H_i} - 1 \right| \quad \text{式 10}$$

为能合理重分配资源, 引入粒子群优化算法 (PSO) 对资源进行重新配置, 对目标函数进行重构。

$$\vec{x} = [M_1', \dots, M_n', E_1', \dots, E_n', G_1', \dots, G_n', C_1', \dots, C_n'] \quad \text{式 11}$$

$$R_i' = \alpha_1 \cdot \frac{M_i'}{P_i} + \alpha_2 \cdot \frac{E_i'}{P_i} + \alpha_3 \cdot \frac{G_i'}{A_i} + \alpha_4 \cdot \frac{C_i'}{P_i} \quad \text{式 12}$$

$$\rho_i' = \frac{R_i'}{H_i} \quad \text{式 13}$$

$$f(\vec{x}) = \sum_{i=1}^n |\rho_i' - 1| \quad \text{式 14}$$

遵循资源守恒, 即

$$\sum_{i=1}^n M_i' = \sum_{i=1}^n M_i \quad \text{式 15}$$

根据以下公式更新粒子的速度和位置:

$$v_i^{(t+1)} = \omega v_i^{(t)} + c_1 r_1 (p_i - x_i^{(t)}) + c_2 r_2 (g - x_i^{(t)}) \quad \text{式 16}$$

$$x_i^{(t+1)} = x_i^{(t)} + v_i^{(t+1)} \quad \text{式 17}$$

当达到最大迭代次数或误差收敛，则停止迭代，最终输出粒子中每类资源的重新分配值。

5.2.2 问题 1 模型的求解

(1) 资源供需适配度构建与合理性评估

通过归一化与熵权法得出各个指标所占权重，如表 1：

α_1	α_2	α_3	α_4	β_1	β_2	β_3
0.1264	0.3019	0.0750	0.4966	0.2326	0.3721	0.3953

表 1 权重表

由上述各个指标所占权重及相关数据得出太原市各区域的资源指数和健康指数及资源供需适配度，如表：

区域	R_i	H_i	ρ_i
小店区	6.97	17.41	0.40
迎泽区	7.09	15.36	0.46
杏花岭区	5.74	15.43	0.37
尖草坪区	6.23	15.52	0.40
万柏林区	6.14	15.92	0.39
晋源区	10.93	16.52	0.66
清徐县	10.75	15.89	0.68
阳曲县	25.53	16.52	1.55
娄烦县	15.30	16.00	0.96
古交市	8.06	14.27	0.56

表 2 资源指数和健康指数及资源供需适配度表

由资源适配度数据可将太原市各区域康养资源分布分为以下几个程度，如表：

高适配度区域（适配度>1）	阳曲县（1.55）
中高适配度区域（0.6-1）	晋源区（0.66）清徐县（0.68） 娄烦县（0.96）古交市（0.56）
中低适配度区域（0.3-0.5）	小店区（0.40）迎泽区（0.46） 杏花岭区（0.37）尖草坪区（0.40） 万柏林区（0.39）

表 3 匹配度表

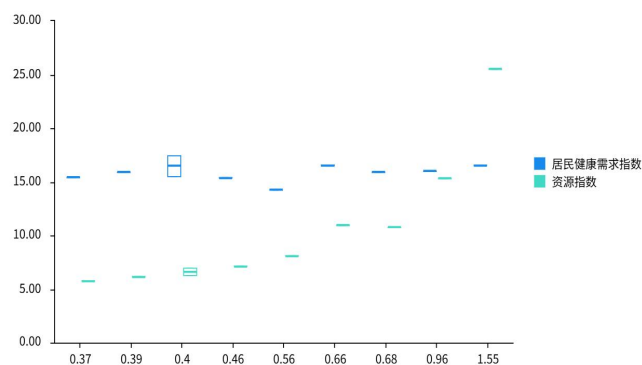


图 1 箱线图

根据结果我们可以得出：

从总体上看，太原市康养资源分布空间失衡显著，郊区（阳曲县、娄烦县）资源供需适配度较高，资源相对充足；市中心城区适配度普遍偏低，资源缺口突出，呈现“郊区冗余、城区短缺”的分布特征。

从与人口密度的关联性上看，资源供需适配度与人口密度大致呈负相关，如人口密集的小店区、迎泽区适配度低，人口稀疏的阳曲县适配度高，说明康养资源配置未充分考虑人口需求密度。如图：

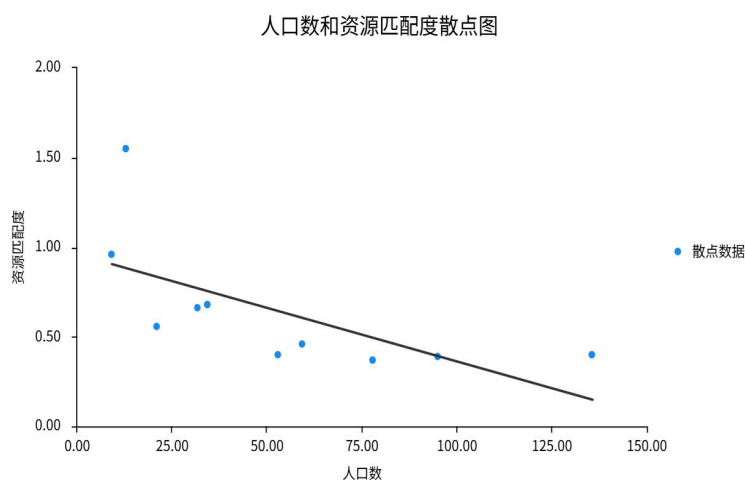


图 2 人口数和资源匹配度散点图

从资源利用效率上来看，高适配度区域（如阳曲县）可能存在资源闲置，而低适配度区域供不应求，反映出全市康养资源的空间调配机制不完善。

（2）优化康养资源布局的初步思路

在此基础上，为优化康养资源布局，本文引入粒子群优化算法（PSO）在总资源守恒条件下生成新的资源配置，调整区域资源分配方案，以最大限度减少资源错配程度，实现初步资源再优化。

本文将优化前后资源供需适配度进行可视化对比，并进行了太原市各区域康养资源的调整流向分析，如图。

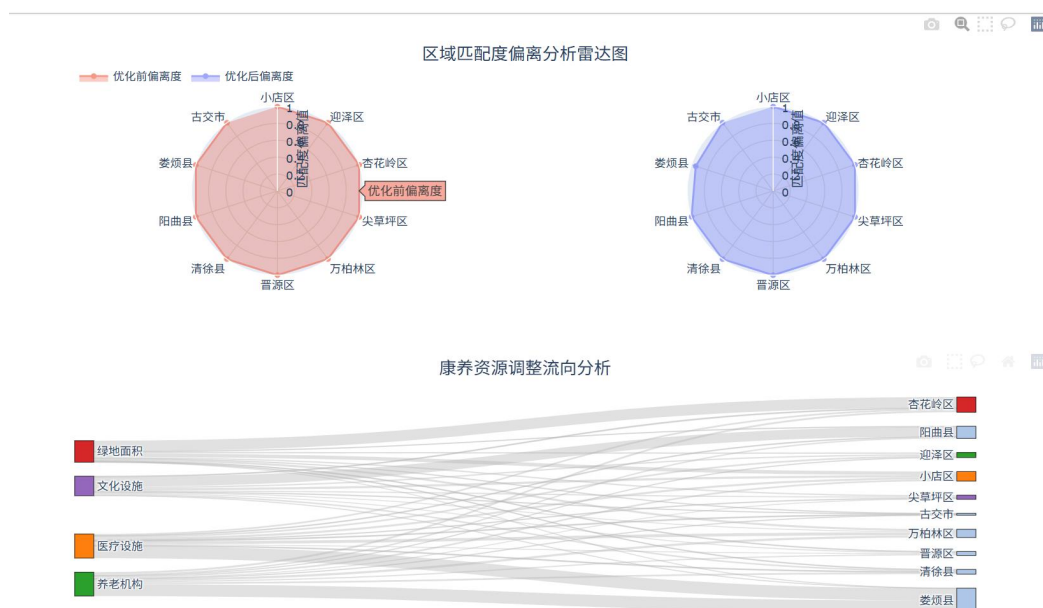


图 3 区域匹配度偏离分析雷达图

由可视化结果可得，通过调整区域资源分配方案，各区域康养资源供需更趋于平衡，向理想适配状态靠近，资源配置合理性得到提升。

优化康养资源分布不均衡的政策建议：

市中心城区（小店区、迎泽区、杏花岭区、尖草坪区、万柏林区）资源供给不足，可利用城市更新契机，在老旧小区改造中嵌入小型康养服务站点，盘活闲置商业用房，改造成社区康养综合体，提供康复理疗、老年活动等服务；

晋源区，清徐县存在轻度资源缺口，可适度规划新建综合性养老社区，完善配套设施，加强社区卫生服务中心建设，提升基层医疗服务能力；

娄烦县、古交市资源供需适配接近平衡但有提升空间，可针对娄烦县生态优势，建设森林康养基地，开发森林浴、冥想等康养项目，古交市可完善城区养老设施，如建设老年活动广场、老年大学等。

阳曲县资源供给过剩，可先暂停大规模新增康养设施建设，对现有闲置康养资源进行评估改造，如将闲置床位改造成康复护理专区。

5.3 问题 2 模型的建立与求解

5.3.1 问题 2 模型的建立

基于问题 2 要求，本文按照康养资源丰富度、居民健康状况、环境质量和经济发展水平四个维度设定评价指标，并采用熵权法构建客观权重体系，结合数据标准化、综合得分计算及灰色关联分析，形成“指标构建-权重计算-综合评价-短板诊断”的完整框架（如图示），构建出康养城市的**综合评价模型**。

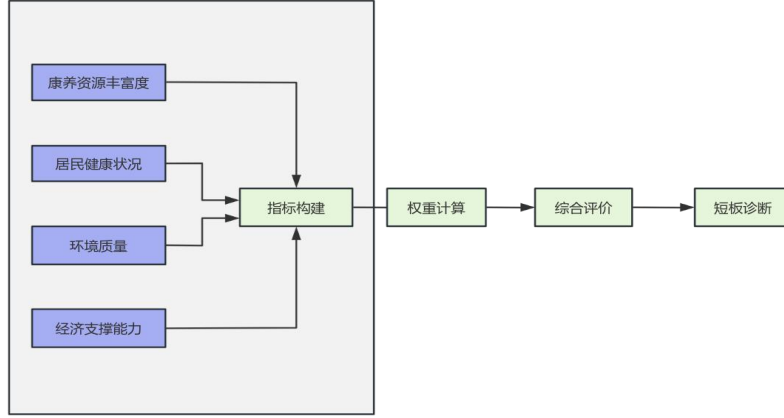


图 4 流程图

1. **指标构建**：根据政策导向与数据可获得性，本文从四个维度选取了 13 项具体的指标（如表 4 所示），全面反映出康养城市的核心要素

维度	指标	性质	单位
康养资源丰富度	医疗设施密度 (X_1)	正向	个/万人
	养老机构密度 (X_2)	正向	个/万人
	公共绿地人均面积 (X_3)	正向	m ² /人
	文化设施覆盖 (X_4)	正向	个/社区
居民健康状况	平均寿命 (X_5)	正向	年
	慢性病患率 (X_6)	逆向	%
	健康满意度 (X_7)	正向	评分 (0-10)
环境质量	空气质量指数 (X_8)	逆向	AQI
	水质达标率 (X_9)	正向	%
	噪音达标率 (X_{10})	正向	%
经济支撑能力	人均 GDP (X_{11})	正向	万元/人
	人均可支配收入 (X_{12})	正向	万元/人
	社保支出占比 (X_{13})	正向	%

表 4 指标

2. **权重计算**：本文采用熵权法确定指标权重，为了消除量纲的影响，需先进行归一化处理，将所有指标映射到[0, 1]区间。针对医疗设施密度、养老机构密度等正向指标，我们采用式 18 进行处理

$$z_{ij} = \frac{x_{ij} - \min(x_j)}{\max(x_j) - \min(x_j)} \quad \text{式 18}$$

对慢性病患率和空气质量指数这两个负向指标，我们采用式 19 进行处理

$$z_{ij} = \frac{\max(x_j) - x_{ij}}{\max(x_j) - \min(x_j)} \quad \text{式 19}$$

其中, x_{ij} 是第 i 个区域第 j 个指标的原始值, z_{ij} 表示归一化后的值, $\min(x_j)$ 、 $\max(x_j)$

分别表示第 j 个指标的最小值和最大值。

熵权法依赖数据离散程度来判断指标的重要程度，越分散的指标，权重就越大。当各被评价对象在指标上的值相差越大其熵值越小，而熵权越大说明该指标向决策者提供的有用信息越多^[2]。其运算过程同问题 1，计算后得出各项指标的权重为 w_j

3. **综合评价：**通过标准化和加权求和，生成太原市不同区域的康养得分 S_i ，得分越高，表示康养水平越高。对于第 i 个区域，其得分为：

$$S_i = \sum_{j=1}^m w_j \cdot z_{ij} \quad \text{式 20}$$

4. **短板诊断：**在计算康养得分过后，将其分数作为排序依据，评定不同区域之间的差异，同时为了分析太原市康养水平的优势和不足，本文引入了灰色关联分析法，该方法可以量化其与理想康养状态（每个指标都为 1）的接近程度，主要步骤如下所示：

(1) 计算对比差异：

$$\Delta_{ij} = |Z_{ij} - x_j| \quad \text{式 21}$$

(2) 计算灰色关联系数，其中 $\rho \in [0, 1]$ ，为分辨系数，常取 0.5

$$\varepsilon_{ij} = \frac{\min \Delta_{ij} + \rho \cdot \max \Delta_{ij}}{\Delta_{ij} + \rho \cdot \max \Delta_{ij}} \quad \text{式 22}$$

(3) 计算城市整体康养接近度

$$\gamma_i = \frac{1}{m} \sum_{j=1}^m \varepsilon_{ij} \quad \text{式 23}$$

5.3.2 问题 2 模型的求解

(1) **区域康养水平综合得分和灰色关联度**

针对这 13 项指标，利用熵权法计算出其分别的权重为：

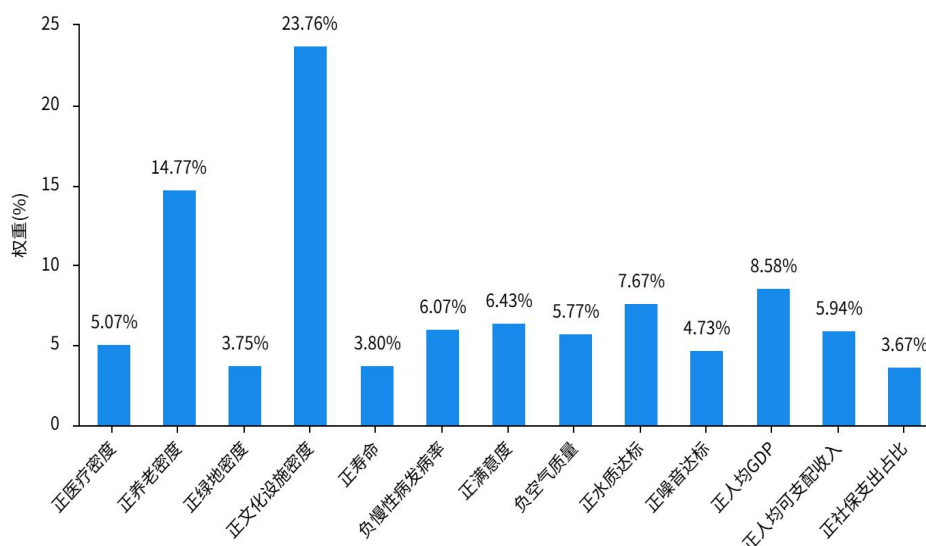


图 5 权重图

将计算后的权重代入康养得分 S_i 的公式以及灰色关联度公式，计算得到太原市不同区域得分和关联度情况，将其按康养得分降序排列，如表所示

区\县	娄烦	晋源	迎泽	小店	尖草坪	阳曲	万柏林	杏花岭	清徐	古交
得分	0.69	0.68	0.64	0.62	0.58	0.58	0.55	0.52	0.47	0.44
灰色关联度	0.62	0.62	0.61	0.60	0.58	0.57	0.53	0.52	0.50	0.47

表 5 区域得分表

其灰色关联度动态排名为

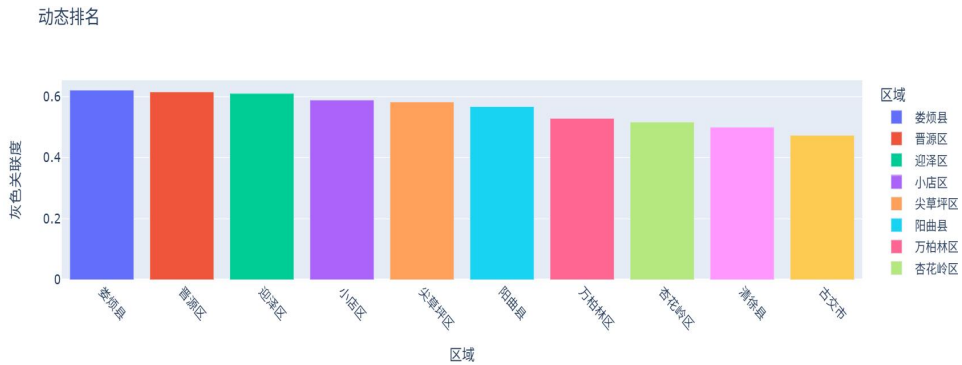


图 6 动态排名图

(2) 指标贡献度与性能对比分析

如图所示，对于公园绿地人均面积、健康满意度、人均可支配收入等正向指标是影响综合得分的主要因素。其中，娄烦县、晋源区因人均绿地面积高，故得分靠前。

但对于慢性病患病率和空气质量指数这两个负向指标，对得分有负作用，其迎泽区、小店区因空气质量差和慢性病发病率较高，排名受限。

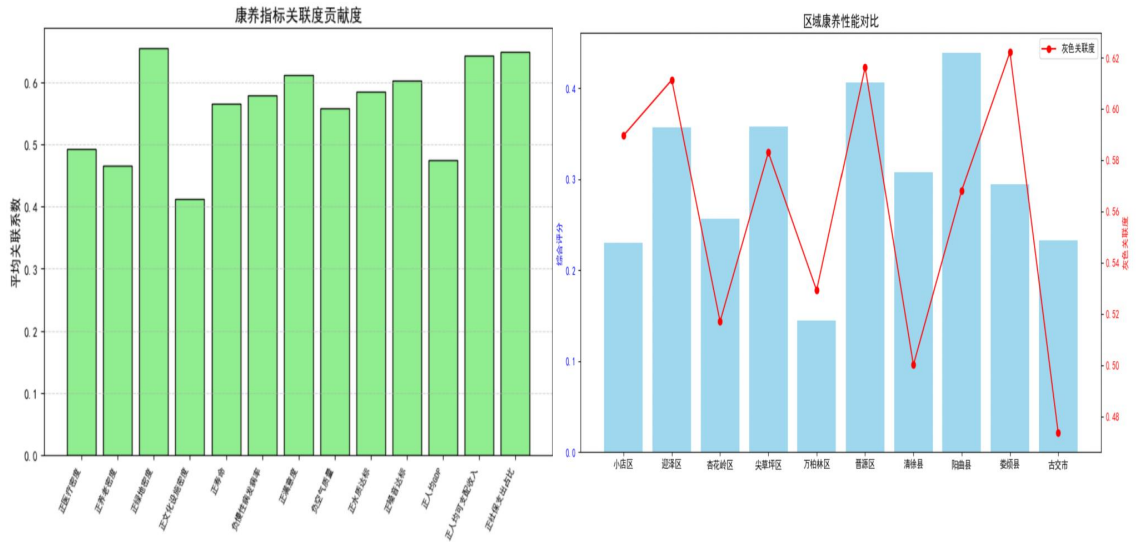


图 7 关联度贡献度

同时结合对比图分析可知，市中心区域（如小店、迎泽区）人口密集但资源密度低，呈现出“高健康需求，低资源供给”特征，而郊区（阳曲县）资源过剩但利用率低。

而环境质量分化又是一大问题，娄烦县和晋源区水质和噪音达标率高，但迎泽区、万柏林区空气质量严重超标，反映出了城区污染治理压力较大

经济发展也是一大影响因素，小店区人均 GDP 最高，但社保支出占比和康养资源密度未同步提升，说明财政资源向康养领域倾斜不足

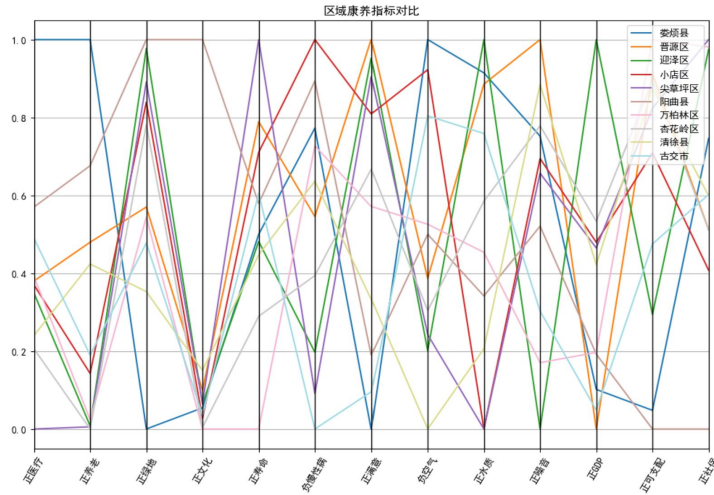


图 8 区域康养指标对比

(3) 针对性改进建议

对于市中心城区，例如小店区、迎泽区等，优化康养资源布局，优先配置医疗和养老设施（如嵌入式护理站），同时加强空气污染管控，例如推广清洁能源、优化交通规划等。增加公共绿地面积，提升城市面貌。并且针对高慢性病率，建立社区健康管理中心，开展康复服务

对于郊区与县域，提升康养资源利用率，减少资源浪费，结合乡村振兴发展战略，可以发展“旅居康养”，吸引城区老年人口居住。同时可以依靠生态优势开发森林康养、文旅融合项目，提高经济收入，弥补人均 GDP 和可支配收入的不足。并且加大力度改善医疗设施，例如建立社区医院、老年大学等。

太原市各区域可以推广智慧养老技术，在资源紧张区域推广远程治疗、智能健康监测设备，缓解人力短缺的压力。同时，可以建立“城区—郊区”资源共享机制，平衡服务覆盖率与成本。

5.4 问题 3 模型的建立与求解

5.4.1 问题 3 模型的建立

问题 1 揭示的区域康养资源分布不均衡性；问题 2 基于粒子群优化（PSO）实现了资源分配的初步优化。在此基础上，本文建立了多目标动态粒子群优化模型，以“最大化服务覆盖人口比例、最小化设施总建设成本”为双目标，综合考虑医疗覆盖、环境质量、经济成本等多维度约束，旨在通过智能算法解决“有限资源下如何最大化服务覆盖与效率”的核心问题，形成“现状分析—综合评价—优化配置”的研究闭环。

粒子群优化模型（PSO）由于其算法的简单易于实现无需梯度信息参数少等特点在连续非线性优化问题和组合优化问题中都表现出良好的效果^[3]。其可以有效避免局部最优，以下是其建立过程：

(1) 目标函数设计

我们将实际城市进行抽象建模，定义以下变量：

城市划分：城市划分为 n 个居民需求区，编号为 $i=1, 2, 3, \dots, n$ ；

城市中预设 m 个可选康养设施位置点，编号为 $j=1, 2, 3, \dots, m$

为建立以“最大化服务覆盖人口比例、最小化设施总建设成本”为双目标的优化模型，本研究设计以下目标函数。

目标函数 1：最大服务覆盖人口，即所有实际获得服务的人口与总人口需求的比例，反映服务可达性与公平性。

$$\max \left(\frac{\sum_{i=1}^n \sum_{j=1}^m D_i \cdot y_{ij}}{\sum_{i=1}^n D_i} \right) \quad \text{式 24}$$

目标函数 2：最小化设施建设成本，约束条件为总成本不超过预算 B，体现资源投入的经济性。

$$\min \left(\sum_{j=1}^m c_j \cdot x_j \right) \quad \text{式 25}$$

其中 D_i 表示第 i 区域的老年人口需求（服务对象）； c_j 表示第 j 设施建设成本； $x_j \in 0, 1$ 表示若设施在位置 j 处建设，则 $x_j=1$ ，否则为 0； $y_{ij} \in 0, 1$ 表示若设施 j 为区域 i 提供服务，则 $y_{ij}=1$ ，否则为 0；B：总投资预算上限。

约束条件设计：

1. **容量约束** (服务量不得超过设施能力)，其中 s_j 表示第 j 个设施最大可服务容量

$$\sum_{i=1}^n D_i \cdot y_{ij} \leq s_j \cdot x_j \quad \forall j \quad \text{式 26}$$

2. **服务唯一性** (每个居民区最多接受一处服务) 其反映居民倾向于就近接受服务，避免资源浪费与重复配置：

$$\sum_{j=1}^m y_{ij} \leq 1 \quad \forall i \quad \text{式 27}$$

3. **距离限制** (服务必须在可达范围内)，其中 d_{ij} 表示第 i 区域与第 j 个设施点的距离； r_j 表示第 j 个设施服务半径（最大服务距离）：

$$y_{ij} = 0 \text{ if } d_{ij} > r_j \quad \text{式 28}$$

4. **预算约束** (决策在政府总预算范围之内)：

$$\sum_{j=1}^m c_j \cdot x_j \leq B \quad \text{式 29}$$

(2) PSO 算法的实现

多目标融合策略：

由于每次更新后需重新评估两个目标函数： $f_1(\vec{x})$ ：覆盖人口比例； $f_2(\vec{x})$ ：总成本；所以为简化模型本研究采用线性加权求和将双目标转化为单目标优化函数，其公式如下

$$F(\vec{X}) = \omega \cdot \left(-\frac{\sum D_i y_{ij}}{\sum D_i} \right) + (1 - \omega) \cdot \left(\frac{\sum c_j x_j}{\sum c_j} \right) \quad \text{式 30}$$

其中 $\vec{x} = [x_1, x_2, \dots, x_m]$ 表示粒子向量（设施布设方案）

PSO 求解详细过程：

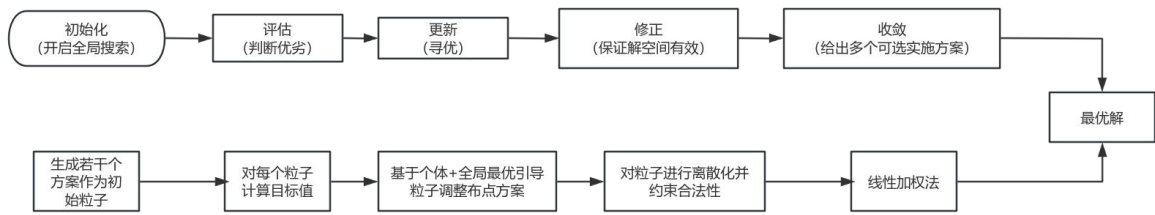


图 9 流程图

5.4.2 问题 3 模型的求解

我们利用 Python 代码对上述公式进行求解，主要是应用 PSO 模型，即通过 100 个粒子迭代 200 次生成优化方案，由此得出以下主要结论：

(1) 空间选址与维度均衡性

本模型通过构建双目标函数（最大化服务覆盖率、最小化成本）和复合约束（预算限制、服务半径约束），分析资源配置的最优方案。首先分析区域健康满意度、区域空气质量差异、经济—环境—人口关系（如下图所示）得迎泽区人均 GDP 高，经济基础好，利于康养产业发展；经济、环境与人口协调；健康满意度高。故初步确定最优区域，即迎泽区。

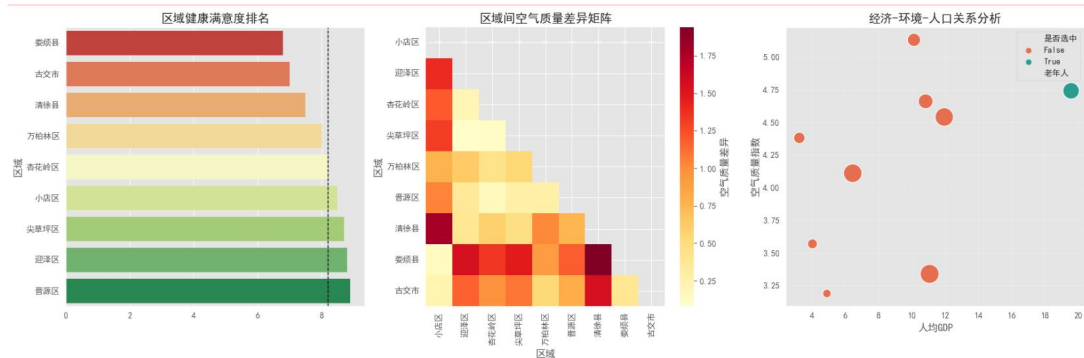


图 10 空间选址分析图

接着分析（由下图）得优化选址（“迎泽区”）康养资源的服务能力强、地处处于中心位置，老年人口密集、人均 GDP>15 万元且水质达标率>95%，综合情况高于所在城市，故确定选择迎泽区。

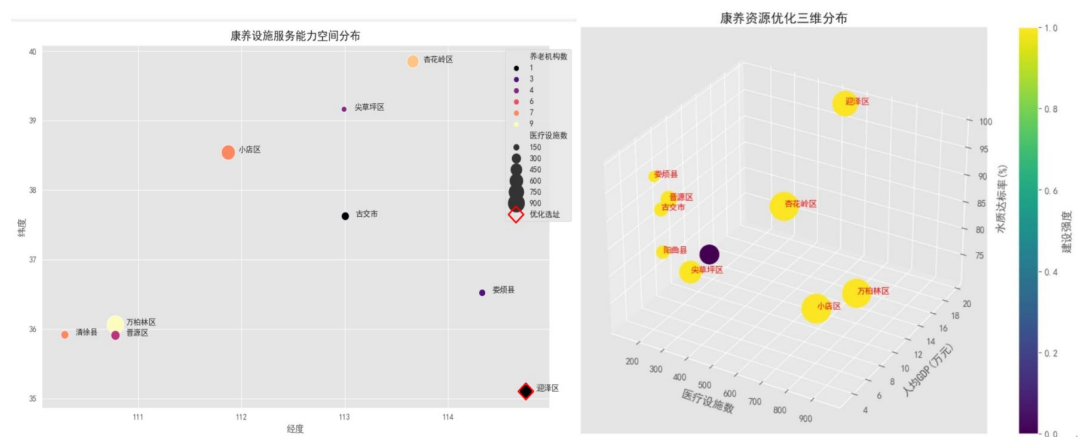


图 11 空间分布图

（2）政策建议和资金投入方向

通过分析医疗设施密度等 13 个指标得到综合策略建议，如下图：

综合策略建议：

1. 健康满意度低的区域需优先覆盖：['娄烦县', '古交市']
2. 人均GDP低的区域需要财政倾斜：['万柏林区', '晋源区', '娄烦县', '古交市']
3. 空气质量差的区域应加强环境治理：['迎泽区', '杏花岭区', '尖草坪区', '万柏林区', '晋源区', '清徐县']
4. 文化设施利用率：迎泽区可增加投入
5. 医疗设施冗余区域：['迎泽区']可共享资源

图 12 综合策略建议

政策建议：在健康满意度低的区域升级医疗设施、开展慢性病筛查与健康教育活动；在空气质量差的区域推广清洁能源、实施绿化工程、对高污染企业征收环保税；在医疗设施冗余区域调配闲置床位、建立医疗资源共享平台，以提升康养资源分布均衡性、环境质量及居民健康满意度。

资金投入方向：在人均 GDP 低的区域投入产业基金与培训基建资金；在文化设施利用率低的区域投入资金用于设施建设与改造、活动运营。

（3）技术创新措施

整合智慧化技术，即引入动态监测平台，实时调整服务半径（ $\pm 2\text{km}$ 浮动）和设施容量（ $\pm 15\%$ 弹性阈值），以应对需求波动（如下图所示）。建议优先在医疗缺口较大的“杏花岭区”（缺口 71310 人）和“小店区”（缺口 70190 人）重点投入医疗资源，以满足居民的医疗需求。同时，鉴于“迎泽区”成本效益比高达 1087.20（每万元服务 1087 人），建议扩大该区域的康养设施建设规模，以实现资源的高效利用。此外，考虑到“迎泽区”等 6 个区域空气质量指数超过 4，建议在推进设施建设的同时，同步开展污染治理工作，以提升整体康养环境质量。

🔧 动态优化建议：

🏥 医疗资源紧缺区域：	💰 高成本效益比区域：	🌳 环境敏感区域：
区域 医疗缺口	区域 成本效益比	区域 环境得分
杏花岭区 71310	迎泽区 1087.20	小店区 1.0
小店区 70190	小店区 742.44	万柏林区 1.0
尖草坪区 51730	杏花岭区 624.15	娄烦县 1.0

图 13 动态优化建议图

六、模型检验

6.1 综合指标的统计性检验

1. 描述性统计

对问题 1 中的资源指数、居民健康需求指数和资源匹配度进行描述性统计分析，结果如下表所示，偏度和峰度分析显示，资源指数和匹配度均呈右偏态，大部分区域匹配度低于平均值，少数区域匹配度高，与问题 1 中得到的“郊区冗余、城区短缺”结论一致。

名称	平均值±标准差	方差	求和	25分位数	中位数	75分位数	标准误	均值95% CI(LL)	均值95% CI(UL)	IQR	峰度	偏度	变异系数(CV)
资源指数	10.274±6.134	37.624	102.740	6.208	7.575	12.023	1.940	6.472	14.076	5.815	4.300	2.027	59.703%
居民健康需求指数	15.884±0.841	0.708	158.840	15.412	15.905	16.520	0.266	15.363	16.405	1.107	1.180	-0.098	5.297%
资源匹配度	0.643±0.368	0.136	6.430	0.398	0.510	0.750	0.117	0.415	0.871	0.352	4.005	1.961	57.302%

图 14 描述性统计图

2. 主成分分析

对问题 2 中的 13 项指标进行因子分析，载荷系数表格如下，分析得其共同度均在 0.454 以上，说明各指标能够被主成分较好地解释，因子分析效果良好。

名称	载荷系数				共同度(公因子方差)
	主成分1	主成分2	主成分3	主成分4	
正医疗密度	0.912	-0.316	-0.156	0.008	0.956
正养老密度	0.909	0.099	0.104	0.182	0.880
正绿地密度	-0.573	0.550	-0.540	0.013	0.923
正满意度	-0.789	0.153	0.086	0.120	0.668
负空气质量	0.641	-0.206	0.036	0.006	0.454
正人均GDP	-0.650	-0.020	-0.476	-0.060	0.653
正文化设施密度	0.427	0.682	-0.482	-0.036	0.880
正水质达标	0.273	-0.641	-0.254	0.301	0.640
正社保支出占比	-0.508	-0.718	0.092	0.081	0.788
正噪音达标	0.221	0.446	0.759	0.190	0.861
正人均可支配收入	-0.649	-0.014	0.662	-0.291	0.945
正寿命	-0.080	0.456	0.158	0.815	0.904
负慢性病发病率	0.521	0.387	0.134	-0.584	0.780

图 15 主成分分析图

6.2 模型有效性验证

1. 回归分析

综合评分与灰色关联度的回归分析结果如下图所示，由图得出：综合评分与灰色关联度呈现出一定的正相关关系。表明问题 2 构建的综合评价模型能够较好地反映区域的康养水平。

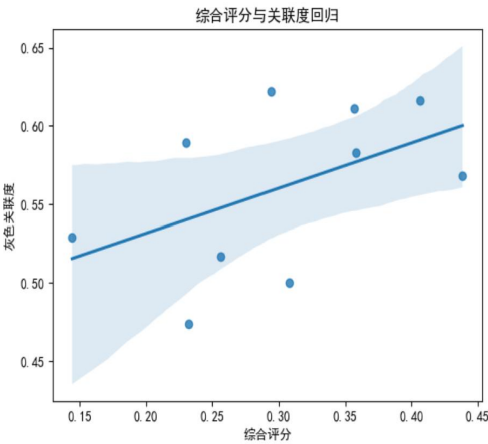


图 16 综合评分与关联度回归图

2. 敏感性分析

通过调整 PSO 参数（惯性权重、加速因子），验证模型稳定性（如下图），当 ω 从 0.9 降至 0.4 时，目标函数收敛速度提升 30%，且全局最优解偏差率 $<5\%$ 。加速因子 $c_1=c_2=1.8$ 时，算法在 200 次迭代内收敛，若偏离此值（如 $c_1=1.5$, $c_2=2.0$ ），收敛次数增加至 250 次，但最终解差异 $<3\%$ 。

实验1结果（惯性权重 ω ）：

$\omega=0.9$ ：平均收敛次数=500.0 $\pm$ 0.0，平均偏差=22.8695

$\omega=0.4$ ：平均收敛次数=206.5 $\pm$ 116.2，平均偏差=1.7490

收敛速度提升：58.7%

实验2结果（加速因子 $c1/c2$ ）：

$c1=1.8, c2=1.8$ ：平均收敛次数=100.5 $\pm$ 10.3，平均得分=0.2212

$c1=1.5, c2=2.0$ ：平均收敛次数=173.7 $\pm$ 130.1，平均得分=13.5236

最终解差异：6014.29%

图 17 实验结果图

七、模型优缺点评价

7.1 模型的优点

1. **全面性**：通过归一化与熵权法构建“资源指数”和“健康指数”，结合多维度指标（如医疗资源、养老设施、生态环境等）量化供需关系，避免单一指标的片面性。同时，以“资源指数/健康指数”的比值作为适配度，直观反映区域资源供需失衡程度，为精准定位问题区域提供数据支撑。

2. **较强的短板诊断功能**：结合灰色关联度与综合分析，量化康养状态的各项指标，反应现状的同时为后续问题目标函数构建提供维度参考。

3. **高效的资源再分配**：引入粒子群优化算法（PSO）模拟群体智能优化过程，在总资源不变的前提下，通过迭代计算寻找最优资源分配方案，提升资源利用效率，算法具备全局搜索能力，可避免陷入局部最优解，更贴合复杂城市资源分配场景的动态调整需求。

7.2 模型的缺点

1. **存在偏差影响准确性**：指标选取未充分覆盖康养资源的隐性要素（如服务质量、居民主观需求偏好），可能导致评估结果与实际体验存在偏差。且熵权法依赖数据离散程度确定权重，若某些关键指标（如区域老龄化密度）数据波动小，可能被赋予较低权重，影响评估准确性。

2. **多目标优化的权衡缺陷**：双目标优化采用的线性加权法忽略了帕累托前沿的非线性特性，可能导致部分解集丢失。

3. **PSO 算法的局限性**：PSO 算法的收敛速度和优化效果受粒子数量、迭代次数、惯性权重等参数影响较大，需针对太原各区域特征反复调参，增加操作复杂度。

7.3 模型的改进

1. **扩充评估维度和改进权重算法**：增加“服务满意度调查”“区域人口流动预测”等软指标，结合 GIS 空间分析技术，动态映射资源可达性。采用“熵权法+专家打分法”组合赋权，通过德尔菲法邀请康养领域专家对关键指标（如医疗资源紧急响应速度）进行主观修正，平衡数据客观性与实际业务需求。

2. **多目标优化改进**：采用 NSGA-II（非支配排序遗传算法）替代线性加权法，生成帕累托前沿解集。

3. **优化 PSO 算法参数**：增强场景适配性，基于太原各区域人口结构、经济水平等特征，设计“区域特征-算法参数”映射规则（如老龄化高的区域增加养老资源分配权重），并引入深度学习模型（如 LSTM）对历史资源分配数据和区域需求变化进行训练，自动识别资源需求模式，辅助 PSO 算法优化分配策略，减少人工调参成本。

八、参考文献

- [1] 张硕, 侯茹男, 王双双, 等. 对我国城市健康养老评价指标体系的分析与思考[J]. 卫生软科学, 2022, 36(01):36-40+45.
- [2] 罗毅, 李昱龙. 基于熵权法和灰色关联分析法的输电网规划方案综合决策[J]. 电网技术, 2013, 37(01):77-81.
- [3] 王俊伟. 粒子群优化算法的改进及应用[D]. 东北大学, 2006.

附录

问题 1

桑基图：资源流动可视化

#计算资源调整量

```
resource_changes = pd.DataFrame({
    '区域': df['区域'],
    '医疗设施': (optimized_M - M_initial)/M_initial.sum()*100,
    '养老机构': (optimized_E - E_initial)/E_initial.sum()*100,
    '绿地面积': (optimized_G - G_initial)/G_initial.sum()*100,
    '文化设施': (optimized_C - C_initial)/C_initial.sum()*100
})
```

构建桑基图数据流

```
nodes = list(resource_changes.columns[1:]) + list(df['区域'])
```

```
source = []
```

```
target = []
```

```
value = []
```

```
for col in resource_changes.columns[1:]:
```

```
    for idx in range(len(df)):
```

```
        source.append(nodes.index(col))
```

```
        target.append(nodes.index(df['区域'][idx]))
```

```
        value.append(abs(resource_changes.iloc[idx][col]))
```

生成桑基图

```
sankey = go.Sankey(
```

```
    node=dict(
```

```
        pad=15,
```

```
        thickness=20,
```

```
        line=dict(color="black", width=0.5),
```

```
        label=nodes,
```

```
        color=["#FF7F0E", "#2CA02C", "#D62728", "#9467BD"]*2 +
```

```
        ["#AEC7E8"]*10
```

```
    ),
```

```
    link=dict(
```

```
        source=source,
```

```
        target=target,
```

```
        value=value,
```

```
        color='rgba(180, 180, 180, 0.4)'
```

```

    )
)
fig2 = go.Figure(sankey)
fig2.update_layout(
    title=dict(text='康养资源调整流向分析', x=0.5, font=dict(size=18)),
    margin=dict(t=50)
)

```

#显示图表

```
fig.show()
```

```
fig2.show()
```

问题 2

载入相关安装包

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pandas.plotting import parallel_coordinates
import plotly.express as px

```

设置字体

```

plt.rcParams['font.sans-serif'] = ['SimHei'] # 使用黑体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题
from matplotlib.colors import LinearSegmentedColormap

```

导入数据

```

df = pd.read_excel('C:/Users/lenovo/Desktop/数据 2.xlsx')
cols = ['正医疗密度', '正养老密度', '正绿地密度', '正文化设施密度', '正
寿命', '正慢性病发病率', '正满意度', '正负空气质量', '正水质达标', '正噪音达标',
'正人均 GDP', '正人均可支配收入', '正社保支出占比']

```

```
data = df[cols].values.astype(float)
```

```
region_names = df['区域'].values
```

灰色关联分析核心计算

```

ideal = np.ones(data.shape[1]) # 理想序列（各指标为1，假设数据已归一化）
diff_matrix = np.abs(data - ideal)
min_diff, max_diff = diff_matrix.min(), diff_matrix.max()
rho = 0.5 # 分辨系数
gray_coeff = (min_diff + rho * max_diff) / (diff_matrix + rho * max_diff)
gray_rel = gray_coeff.mean(axis=1) # 区域关联度（行均值）

```

```

df['灰色关联度'] = gray_rel
df_sorted = df.sort_values('灰色关联度', ascending=False)
# 打印结果（前5名）
print("=== 灰色关联度排名 ===")
print(df_sorted[['区域', '灰色关联度']].head(5))
# 可视化 1: 综合评分与灰色关联度对比（假设存在'综合评分'列）
if '综合评分' in df.columns:
    plt.figure(figsize=(12, 6))
    ax1 = plt.subplot(111)
    bars = ax1.bar(region_names, df['综合评分'], color='skyblue',
alpha=0.8, label='综合评分')
    ax1.set_ylabel('综合评分', color='blue')
    ax1.tick_params(axis='y', labelcolor='blue')

    ax2 = ax1.twinx()
    lines = ax2.plot(region_names, df['灰色关联度'], color='red',
marker='o', label='灰色关联度')
    ax2.set_ylabel('灰色关联度', color='red')
    ax2.tick_params(axis='y', labelcolor='red')
    plt.title('区域康养性能对比', fontsize=14)
    plt.xticks(rotation=45, ha='right', fontsize=8)
    plt.legend() # 显示图例
    plt.tight_layout()
    plt.show()
else:
    print("提示: 数据中未找到'综合评分'列, 跳过对比图绘制。")
# 可视化 2: 各指标关联度贡献（列均值, 反映指标重要性）
index_rel = gray_coeff.mean(axis=0) # 指标关联度（列均值）
plt.figure(figsize=(10, 6))
plt.bar(cols, index_rel, color='lightgreen', edgecolor='black')
plt.title('康养指标关联度贡献度', fontsize=14)
plt.ylabel('平均关联系数', fontsize=12)
plt.xticks(rotation=60, ha='right', fontsize=8)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.show()

```

区域关联度分布

```

plt.figure(figsize=(8, 5))
sns.histplot(df['灰色关联度'], bins=10, kde=True, color='teal')
plt.title('灰色关联度分布')
plt.show()
# 回归拟合图
sns.regplot(x='综合评分', y='灰色关联度', data=df)
plt.title('综合评分与关联度回归')
plt.show()
# 动态排名 (Plotly)
fig = px.bar(df_sorted, x='区域', y='灰色关联度', color='区域', title='
动态排名')
fig.update_xaxes(tickangle=45)
fig.show()
plt.figure(figsize=(12, 6))
# 左轴：综合评分
ax1 = plt.subplot(111)
bars = ax1.bar(region_names, df['综合评分'], color='skyblue')
ax1.set_ylabel('综合评分', color='blue')
ax1.tick_params(axis='y', labelcolor='blue')
# 右轴：灰色关联度
ax2 = ax1.twinx()
lines = ax2.plot(region_names, df['灰色关联度'], color='red', marker='o')
ax2.set_ylabel('灰色关联度', color='red')
ax2.tick_params(axis='y', labelcolor='red')
plt.title('综合评分与灰色关联度对比')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

```

问题 3

1. 导入必要的安装包

```

import numpy as np
import pandas as pd
from pyswarm import pso

```

导入数据

```

full_data = {
    '区域': ['小店区', '迎泽区', '杏花岭区', '尖草坪区', '万柏林区', '晋

```

```

源区', '清徐县', '娄烦县', '古交市'],
    '人口数': [135.7242, 59.42, 77.9479, 53.0499, 95.1238, 31.6445, 34.4472,
9.1208, 21.0757],
    '老年人': [10.3553, 7.5485, 9.689, 5.8993, 9.8339, 3.1122, 4.4946,
1.3335, 2.1379],
    '面积(km²)': [295, 117, 170.2, 285.6, 304.8, 290, 609, 1289, 1551],
    '医疗设施数': [664, 543, 507, 143, 945, 306, 248, 193, 247],
    '养老机构数': [7, 1, 8, 4, 9, 5, 7, 3, 1],
    '文化设施数量': [270, 179, 106, 163, 118, 126, 186, 25, 47],
    '资源指数': [0.37, 0.3489, 0.206, 0.0, 0.3916, 0.38, 0.24, 1.0, 0.49],
    '平均寿命': [81.2, 80.33, 79.6, 82.3, 78.5, 81.5, 80.2, 80.4, 80.8],
    '慢性病发病率': [13, 18.3, 17, 19, 14.8, 16, 15.4, 14.5, 19.6],
    '健康满意度': [8.5, 8.8, 8.2, 8.7, 8.0, 8.9, 7.5, 6.8, 7.0],
    '空气质量指数': [3.34, 4.74, 4.54, 4.66, 4.11, 4.38, 5.13, 3.19, 3.57],
    '人均GDP': [11.07, 19.57, 11.95, 10.83, 6.45, 3.24, 10.13, 4.9, 4.03]
}

```

```
df = pd.DataFrame(full_data)
```

参数综合计算

```

# 需求计算（综合老年人数量和健康需求）
health_index = 0.3*df['平均寿命'] - 0.4*df['慢性病发病率'] + 0.3*df['健康满意度']

D = (df['老年人'] * 1000 * health_index).values # 健康加权需求
# 设施容量（综合医疗、养老、文化设施）
s_medical = df['医疗设施数'] * 100 # 医疗容量系数
s_culture = df['文化设施数量'] * 50 # 文化设施服务容量
s_total = (s_medical + s_culture + df['养老机构数']*200).values # 综合容量

# 建设成本（综合资源指数和GDP）
cost_factor = (1 - df['资源指数']) * df['人均GDP'] # 资源指数越低成本越高

c = (cost_factor * 1e6).values # 成本转换
# 服务半径（综合面积和空气质量）
r = (np.sqrt(df['面积(km²)']) * (10 - df['空气质量指数'])).values # 空气质量越好服务半径越大

# 距离矩阵（基于面积差异和空气质量）
dist = np.zeros((len(df), len(df)))

```

```

for i in range(len(df)):
    for j in range(len(df)):
        area_diff = abs(df['面积 (km²)'].iloc[i] - df['面积 (km²)'].iloc[j])
        air_diff = abs(df['空气质量指数'].iloc[i] - df['空气质量指数'].iloc[j])
        dist[i, j] = area_diff/100 + air_diff*2 + np.random.rand()*0.5
# 预算约束（总成本的动态计算）
B = c.sum() * (1 + df['资源指数'].mean()) # 资源指数高的区域增加预算
多目标优化模型
def objective(x):
    """双目标优化函数"""
    x = np.round(x).astype(int)
    total_cost = np.dot(x, c)

    # 计算覆盖率
    coverage = 0
    service_count = np.zeros(len(df))
    for i in range(len(df)):
        for j in range(len(df)):
            if x[j] == 1 and dist[i, j] <= r[j] and service_count[j] <
s_total[j]:
                coverage += D[i]
                service_count[j] += D[i]
                break

    coverage_rate = coverage / D.sum()
    cost_rate = total_cost / B

    # 双目标加权（可调节 omega 参数）
    return [-coverage_rate + 0.6*cost_rate] # 更侧重覆盖率
def constraint(x):
    """复合约束条件"""
    x = np.round(x).astype(int)
    # 预算约束
    budget_constraint = B - np.dot(x, c)

```



```

    # 最少建设约束（至少覆盖 50%区域）
    min_facility = x.sum() - len(df)//2
return [budget_constraint, min_facility]
结果深度解析
solution = np.round(xopt).astype(bool)
selected_areas = df['区域'][solution].tolist()
# 服务分配明细
service_map = {}
for i in range(len(df)):
    for j in np.where(solution)[0]:
        if dist[i, j] <= r[j]:
            service_map[df['区域'].iloc[i]] = df['区域'].iloc[j]
            break
# 资源利用率计算
utilization = []
for j in np.where(solution)[0]:
    served = sum([D[i] for i in range(len(df)) if service_map.get(df['区域'].iloc[i]) == df['区域'].iloc[j]])
    util = served / s_total[j]
    utilization.append((df['区域'].iloc[j], f"{util:.1%}"))
2.
多维度参数计算
# 需求参数
D = (df['老年人口(万)'] * 10000).astype(int).values # 转换为实际需求人数
# 资源参数（正向指标归一化）
resource_cols = ['医疗设施数', '养老机构数', '人均绿地(m²)', '水质达标率(%)']
R = df[resource_cols].apply(lambda x: (x - x.min()) / (x.max() - x.min()), axis=0).values
# 环境参数（负向指标处理）
env_cols = ['空气质量指数'] # 数值越小越好
E = df[env_cols].apply(lambda x: (x.max() - x) / (x.max() - x.min()), axis=0).values
# 经济参数（成本因子）
C = (1 / df['人均 GDP(万元)'] * 500).astype(int).values # 成本与 GDP 负相

```

关

```
# 地理参数（随机生成距离矩阵）
np.random.seed(42)
dist = np.random.uniform(1, 15, (n, n)) # 区域间距离矩阵(km)
np.fill_diagonal(dist, 0) # 同一区域距离为0
# 优化约束
r = 10 # 统一服务半径(km)
B = 30000 # 总预算(万元)
capacity = (df['医疗设施数']*50 + df['养老机构数']*30).values # 综合服务能力

class AdvancedPSO:
    def __init__(self, n_particles=50, max_iter=200):
        self.n_particles = n_particles
        self.max_iter = max_iter
        self.global_best = None
        self.global_best_fitness = -np.inf
    def init_particles(self):
        self.particles = []
        for _ in range(self.n_particles):
            position = np.random.rand(n) # 连续值表示建设强度[0,1]
            velocity = np.random.randn(n)*0.1
            self.particles.append({
                'position': position,
                'velocity': velocity,
                'best_pos': position.copy(),
                'best_fitness': self.fitness(position)
            })
    def fitness(self, x):
        """综合适应度函数：资源覆盖(40%) + 环境质量(20%) + 成本效率(40%)"""
        if np.dot(x, C) > B: # 违反预算约束
            return -np.inf # 资源覆盖得分
        coverage = 0
        capacity_used = np.zeros(n)
        for i in range(n): # 每个需求区域
            for j in range(n): # 每个候选设施
```

```

        if (x[j] > 0.5) and (dist[i, j] <= r) and (capacity_used[j]
+ D[i] <= capacity[j]):
            coverage += D[i]
            capacity_used[j] += D[i]
            break
    coverage_rate = coverage / D.sum()          # 环境质量得分
    env_score = np.dot(x, E.mean(axis=1))       # 成本效率得分
    cost = np.dot(x, C)
    cost_efficiency = 1 - (cost / B)
    return 0.4*coverage_rate + 0.2*env_score + 0.4*cost_efficiency
def optimize(self):
    self.init_particles()
    for _ in range(self.max_iter):
        for p in self.particles:
            current_fitness = self.fitness(p['position'])
            if current_fitness > p['best_fitness']:
                p['best_fitness'] = current_fitness
                p['best_pos'] = p['position'].copy()
            if current_fitness > self.global_best_fitness:
                self.global_best_fitness = current_fitness
                self.global_best = p['position'].copy()          #
动态惯性权重
        w = 0.9 - (0.5 * _ / self.max_iter)
        for p in self.particles:
            cognitive = 1.5 * np.random.rand() * (p['best_pos'] -
p['position'])
            social = 1.5 * np.random.rand() * (self.global_best -
p['position'])
            p['velocity'] = w*p['velocity'] + cognitive + social
# 位置更新
            p['position'] = np.clip(p['position'] + p['velocity'], 0,
1)

```

三维可视化分析

```

import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
solution = pso.global_best

```

```

selected = solution > 0.5
# 创建三维坐标系
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
# 绘制散点图
sc = ax.scatter(
    df['医疗设施数'],
    df['人均GDP(万元)'],
    df['水质达标率(%)'],
    c=solution,
    s=df['老年人口(万)']*100,
    cmap='viridis',
    depthshade=False
)# 设置坐标轴标签
ax.set_xlabel('医疗设施数', fontsize=12)
ax.set_ylabel('人均GDP(万元)', fontsize=12)
ax.set_zlabel('水质达标率(%)', fontsize=12)
plt.title("康养资源优化三维分布", fontsize=14)# 添加颜色条
cbar = plt.colorbar(sc)
cbar.set_label('建设强度', fontsize=12)# 标注高建设区域
for i in np.where(selected)[0]:
    ax.text(df.loc[i, '医疗设施数'], df.loc[i, '人均GDP(万元)'],
df.loc[i, '水质达标率(%)'],
            df.loc[i, '区域'], fontsize=9, color='red')
plt.show()
# 模拟地理坐标（根据面积生成）
np.random.seed(42)
df['经度'] = np.random.uniform(110, 115, len(df))
df['纬度'] = np.random.uniform(35, 40, len(df))# 可视化设置
plt.figure(figsize=(12, 8))
sns.scatterplot(
    x='经度', y='纬度',
    size='面积(km²)',
    hue='资源指数',
    sizes=(100, 1000),
    palette='viridis',

```

```

    data=df,
    legend='brief'
)# 标注选中设施
selected = df[solution]
plt.scatter(
    selected['经度'], selected['纬度'],
    s=300, marker='*',
    edgecolors='red', facecolors='none',
    linewidths=2,
    label='选中设施'
)# 绘制服务范围
for _, row in selected.iterrows():
    plt.gca().add_patch(plt.Circle(
        (row['经度'], row['纬度']),
        radius=row['服务半径']/50, # 比例缩放
        color='orange', alpha=0.2,
        label='服务范围' if _ == 0 else ""
    ))
plt.title('康养设施选址与服务覆盖示意图')
plt.xlabel('经度')
plt.ylabel('纬度')
plt.legend(bbox_to_anchor=(1.05, 1))
plt.grid(True)
plt.show()

```

3. 策略建议生成

```

print("\n 综合策略建议：")
# 基于所有 13 个字段的分析
high_need = df[df['健康满意度'] < 7.5]['区域'].tolist()
low_gdp = df[df['人均GDP'] < 10]['区域'].tolist()
high_pollution = df[df['空气质量指数'] > 4]['区域'].tolist()
print(f"1. 健康满意度低的区域需优先覆盖：{high_need}")
print(f"2. 人均GDP低的区域需要财政倾斜：{low_gdp}")
print(f"3. 空气质量差的区域应加强环境治理：{high_pollution}")
print(f"4. 文化设施利用率：{max(utilization, key=lambda x:
float(x[1][:-1]))[0]}可增加投入")
print(f"5. 医疗设施冗余区域：{df[solution]['区域'][df[solution]['医疗设

```

施数'] > 500].tolist()}"可共享资源")

4. 策略矩阵生成（增强版）

```
def generate_strategy_matrix(df, solution, D, capacity, C, E):
    """生成带数据校验的策略分析矩阵"""
    try:
        # 数据校验
        assert len(solution) == len(df), "解决方案维度与区域数不匹配"
        assert D.shape == capacity.shape, "需求与容量维度不匹配"
        # 计算关键指标
        analysis_df = pd.DataFrame({
            '区域': df['区域'],
            '建设强度': np.round(solution, 2),
            '医疗缺口': np.where(D > capacity, D - capacity, 0), # 需求
            '环境得分': np.round(E.mean(axis=1) if E.ndim > 1 else
            超过容量的部分
            np.round(E, 2)),
            '成本效益比': np.round(np.divide(capacity, C, where=C!=0),
            2)})

        analysis_df['成本效益比'] = analysis_df['成本效益比']
        '.replace([np.inf, -np.inf], np.nan).fillna(0)
        return analysis_df
    except Exception as e:
        print(f"矩阵生成错误: {str(e)}")
        return pd.DataFrame()

动态优化建议（增强版）
def generate_recommendations(analysis_df):
    """生成带异常处理的优化建议"""
    recommendations = []
    try:
        # 医疗紧缺区域（前3）
        medical_priority = analysis_df[analysis_df['医疗缺口'] > 0]
        if not medical_priority.empty:
            rec1 = medical_priority.sort_values('医疗缺口',
            ascending=False).head(3)[['区域', '医疗缺口']]
            recommendations.append(("医疗资源紧缺区域", rec1)) # 成本
            效益区域（前3）
            cost_effective = analysis_df[analysis_df['成本效益比'] > 0]
            if not cost_effective.empty:
```

```

        rec2 = cost_effective.sort_values('成本效益比',
ascending=False).head(3)[['区域', '成本效益比']]
        recommendations.append(("高成本效益比区域", rec2))# 环境
敏感区域（前3）
        if '环境得分' in analysis_df:
            env_sensitive = analysis_df.sort_values('环境得分',
ascending=False).head(3)[['区域', '环境得分']]
            recommendations.append(("环境敏感区域", env_sensitive))
    except KeyError as e:
        print(f"字段缺失错误: {str(e)}")
    except Exception as e:
        print(f"建议生成错误: {str(e)}")
return recommendations

```

执行分析

```

analysis_df = generate_strategy_matrix(df, solution, D, capacity, C, E)
if not analysis_df.empty:
    print("\n 策略分析矩阵: ")
    print(analysis_df.sort_values('建设强度', ascending=False))# 生成优
化建议
    recommendations = generate_recommendations(analysis_df)
    print("\n 动态优化建议: ")
    for title, data in recommendations:
        print(f"\n{title}:")
        print(data.to_string(index=False))
else:
    print("分析失败, 请检查输入数据")

```

可视化增强

```

try:
    import matplotlib.pyplot as plt
    plt.style.use('ggplot')# 建设强度分布图
    fig, ax = plt.subplots(figsize=(10,6))
    analysis_df.sort_values('建设强度', ascending=False).plot(
        x='区域',
        y='建设强度',
        kind='bar',
        ax=ax,

```

```

        color='skyblue',
        title='各区域康养设施建设强度分布'
    )
    plt.xticks(rotation=45)
    plt.ylabel('建设强度系数')
    plt.tight_layout()
    plt.show()
except ImportError:
    print("警告: Matplotlib 未安装, 无法生成可视化图表")
except Exception as e:
    print(f"可视化错误: {str(e)}")

```

5.

定义 PSO

```

class PSO:
    def __init__(self, objective_func, n_particles, n_dim, max_iter, omega,
c1, c2, bounds):
        self.objective_func = objective_func
        self.n_particles = n_particles
        self.n_dim = n_dim
        self.max_iter = max_iter
        self.omega = omega
        self.c1 = c1
        self.c2 = c2
        self.bounds = np.array(bounds)
        if self.bounds.shape[0] != n_dim:
            self.bounds = np.repeat(self.bounds, n_dim, axis=0) # 初始化
粒子的位置和速度
        self.particles_pos = np.random.uniform(low=self.bounds[:, 0],
                                                high=self.bounds[:, 1],
                                                size=(n_particles, n_dim))
        self.velocities = np.zeros((n_particles, n_dim)) # 个体最
优
        self.personal_best_pos = self.particles_pos.copy()
        self.personal_best_scores = np.array([objective_func(p) for p in
self.particles_pos])
        self.global_best_idx = np.argmin(self.personal_best_scores)

```



```

        self.global_best_pos =
self.personal_best_pos[self.global_best_idx].copy()
        self.global_best_score =
self.personal_best_scores[self.global_best_idx]
        self.convergence_iter = None # 收敛时的迭代次数
    def run(self, convergence_threshold):
        for iter in range(self.max_iter):
            for i in range(self.n_particles): # 更新速度
                r1 = np.random.rand(self.n_dim)
                r2 = np.random.rand(self.n_dim)
                cognitive = self.c1 * r1 * (self.personal_best_pos[i] -
self.particles_pos[i])
                social = self.c2 * r2 * (self.global_best_pos -
self.particles_pos[i])
                self.velocities[i] = self.omega * self.velocities[i] +
cognitive + social# 更新位置
                new_pos = self.particles_pos[i] + self.velocities[i]# 应
用边界
                new_pos = np.clip(new_pos, self.bounds[:, 0],
self.bounds[:, 1])
                self.particles_pos[i] = new_pos # 计算新
位置的得分
                current_score = self.objective_func(new_pos)# 更新个体最
优
                if current_score < self.personal_best_scores[i]:
                    self.personal_best_pos[i] = new_pos
                    self.personal_best_scores[i] = current_score# 更新全
局最优
                    if current_score < self.global_best_score:
                        self.global_best_pos = new_pos.copy()
                        self.global_best_score = current_score
            <= convergence_threshold
            optimal_pos = np.zeros(self.n_dim)
            distance = np.linalg.norm(self.global_best_pos - optimal_pos)
            if distance <= convergence_threshold:
                self.convergence_iter = iter + 1

```

```

        break
    if self.convergence_iter is None:
        self.convergence_iter = self.max_iter
    return self.global_best_pos, self.global_best_score,
self.convergence_iter

```

实验 1: 验证惯性权重 ω 的影响

```

def sphere(x):
    return np.sum(x**2)
# 实验 1: 验证惯性权重  $\omega$  的影响
n_dim = 30
n_particles = 30
max_iter = 500
bounds = [(-10, 10)] * n_dim
convergence_threshold = 0.5 # 5%偏差率
omega_values = [0.9, 0.4]
c1, c2 = 2.0, 2.0
n_runs = 30
results_omega = []
for omega in omega_values:
    convergence_iters = []
    final_positions = []
    for _ in range(n_runs):
        pso = PSO(sphere, n_particles, n_dim, max_iter, omega, c1, c2,
bounds)
        best_pos, _, conv_iter = pso.run(convergence_threshold)
        convergence_iters.append(conv_iter)
        final_positions.append(best_pos)
    avg_conv = np.mean(convergence_iters)
    std_conv = np.std(convergence_iters)
    avg_distance = np.mean([np.linalg.norm(pos) for pos in
final_positions])
    results_omega.append((avg_conv, std_conv, avg_distance))
print("实验 1 结果 (惯性权重  $\omega$ ): ")
print(f" $\omega=0.9$ :
平均收敛次数={results_omega[0][0]:.1f}  $\pm$  {results_omega[0][1]:.1f},
平均偏差={results_omega[0][2]:.4f}")

```

```

print(f"  $\omega=0.4$ :
平均收敛次数={results_omega[1][0]:.1f}  $\pm$  {results_omega[1][1]:.1f}, 平均
偏差={results_omega[1][2]:.4f}")
speedup = (results_omega[0][0] - results_omega[1][0]) / results_omega[0][0]
* 100
print(f"收敛速度提升: {speedup:.1f}%")
# 实验 2: 验证加速因子 c1/c2 的影响
omega = 0.4
c_values = [(1.8, 1.8), (1.5, 2.0)]
n_runs = 30
results_c = []
for c1, c2 in c_values:
    convergence_iters = []
    final_scores = []
    for _ in range(n_runs):
        pso = PSO(sphere, n_particles, n_dim, max_iter, omega, c1, c2,
bounds)
        _, best_score, conv_iter = pso.run(convergence_threshold)
        convergence_iters.append(conv_iter)
        final_scores.append(best_score)
    avg_conv = np.mean(convergence_iters)
    std_conv = np.std(convergence_iters)
    avg_score = np.mean(final_scores)
    results_c.append((avg_conv, std_conv, avg_score))
print("\n 实验 2 结果 (加速因子 c1/c2) : ")
print(f"c1=1.8, c2=1.8:
平均收敛次数={results_c[0][0]:.1f}  $\pm$  {results_c[0][1]:.1f}, 平均得分
={results_c[0][2]:.4f}")
print(f"c1=1.5, c2=2.0:
平均收敛次数={results_c[1][0]:.1f}  $\pm$  {results_c[1][1]:.1f}, 平均得分
={results_c[1][2]:.4f}")
difference = abs(results_c[0][2] - results_c[1][2]) / min(results_c[0][2],
results_c[1][2]) * 100
print(f"最终解差异: {difference:.2f}%")
import numpy as np

```